

Towards Stricter Black-box Integrity Verification of Deep Neural Network Models

Chaoxiang He*
Huazhong University of Science and Technology
Wuhan, Hubei, China
Shanghai Jiao Tong University
Shanghai, China
hechaoxiang@hust.edu.cn

Xiaofan Bai*
Huazhong University of Science and Technology
Wuhan, Hubei, China
xiaofanbai@hust.edu.cn

Xiaojing Ma†*
Huazhong University of Science and Technology
Wuhan, Hubei, China
lindahust@hust.edu.cn

Bin B. Zhu
Microsoft Corporation
Beijing, China
binzhu@microsoft.com

Pingyi Hu*
Huazhong University of Science and Technology
Wuhan, Hubei, China
pingyihu@hust.edu.cn

Jiayun Fu*
Huazhong University of Science and Technology
Wuhan, Hubei, China
fujiayun@hust.edu.cn

Hai Jin*
Huazhong University of Science and Technology
Wuhan, Hubei, China
hjin@hust.edu.cn

Dongmei Zhang
Microsoft Corporation
Beijing, China
dongmeiz@microsoft.com

Abstract

Cloud-based machine learning services offer significant advantages but also introduce the risk of tampering with cloud-deployed deep neural network (DNN) models. *Black-box integrity verification* (BIV) allows model owners and end-users to determine if a cloud-deployed DNN model has been tampered with by examining only the top-1 label responses. Fingerprinting generates fingerprint samples to query the model, achieving BIV with no impact on the model's accuracy. In this paper, we present *BIVBench*, the first comprehensive benchmark for BIV of DNN models. BIVBench covers 16 types of model modifications, providing extensive coverage of practical modification scenarios. Our analysis reveals that existing fingerprinting methods, which are typically focused on significant tampering, lack the sensitivity needed to effectively detect subtle yet common and potentially severe modifications. To address this limitation, we propose *MiSentry* (Model Integrity Sentry), a novel fingerprinting method that leverages meta-learning. MiSentry strategically incorporates a few subtly modified models into the meta-learning model zoo and maximizes the divergence of output predictions between the target model and the modified models in the model zoo to generate highly sensitive, generalizable, and effective fingerprint samples. Extensive evaluations using BIVBench demonstrate that MiSentry outperforms existing state-of-the-art methods overall and significantly surpasses them in detecting subtle modifications. The BIVBench and supplementary materials are available at: https://github.com/CGCL-codes/BIVBench.

CCS Concepts

• **Security and privacy**; • **Computing methodologies** → **Artificial intelligence**; **Machine learning**;

Keywords

Integrity Verification; Tampering Detection; Fingerprinting; Deep Neural Networks

ACM Reference Format:
Chaoxiang He, Xiaofan Bai, Xiaojing Ma, Bin B. Zhu, Pingyi Hu, Jiayun Fu, Hai Jin, and Dongmei Zhang. 2024. Towards Stricter Black-box Integrity Verification of Deep Neural Network Models. In *Proceedings of the 32nd ACM International Conference on Multimedia (MM '24), October 28-November 1, 2024, Melbourne, VIC, Australia.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3664647.3681691

1 Introduction

Deep neural networks (DNNs) have significantly advanced various computer vision tasks [15, 31, 38]. Major cloud providers like Microsoft Azure ML [32], Google AutoML [13], and Amazon SageMaker [1] offer platforms for users to deploy their models. However, cloud deployment introduces risks. Adversaries may tamper with models via Trojan [30] or backdoor [14] attacks to insert harmful behavior, or competitors might degrade models using bit-flipping attacks [51]. Additionally, cloud providers might replace models with simpler versions to cut costs.

To address these threats, *fingerprinting* [17, 48] and *watermarking* [54] methods are proposed for *black-box integrity verification* (BIV) of DNN classification models. BIV aims to detect tampering by querying the model and comparing returned top-1 category

*Chaoxiang He, Xiaofan Bai, Xiaojing Ma, Pingyi Hu and Jiayun Fu are with National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Hubei Engineering Research Center on Big Data Security, Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology. Hai Jin is with National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology.
†Corresponding author: Xiaojing Ma.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '24, October 28-November 1, 2024, Melbourne, VIC, Australia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0686-8/24/10
https://doi.org/10.1145/3664647.3681691

labels with those of the *target model* (i.e., the unmodified model to protect). Watermarking embeds watermark samples into the model through training or fine-tuning, potentially reducing performance, and cannot generate new watermark samples post-training, making it vulnerable to leaked watermark samples, which can be exploited to evade detection. In contrast, fingerprinting does not alter the model and can generate unlimited fingerprint samples, making it robust to leaked fingerprint samples (see Section 5.5 for details).

Fingerprinting methods are currently evaluated on *significant tampering*, such as backdoor attacks, Trojan, and model pruning. *Subtle tampering*, such as subpopulation data-poisoning attacks [21], which cause the model to mispredict for a small subset of samples, and *benign modifications*, like model updates [12, 41] or fine-tuned versions for different purposes, for which a specific deployed version may need to be verified, are often overlooked. Traditional data integrity verification ensures data remains unaltered, and similarly, DNN integrity verification should detect all modifications, malicious or benign, significant or subtle. This study aims to address the gap by focusing on stricter black-box DNN integrity verification that should detect all observable modifications. A model is said to be *observably modified* if its decision boundary is observably altered, i.e., there exists an example such that its prediction result received by an end user, e.g., top-1 label for a classification model, is different from that of the target model. Intuitively, subtler model modifications are harder to observe and thus harder to detect.

In this paper, we first introduce *BIVBench*, the first benchmark for stricter black-box integrity verification of DNN models. It includes 16 types of *practically observable* model modifications, covering common malicious and benign scenarios, and introduces 10 new types. Our evaluation, reported in Section 3.2, shows that existing fingerprinting methods struggle with subtle modifications.

We then propose *MiSentry*, a novel DNN fingerprinting method for detecting both significant and subtle modifications. It enhances detection sensitivity by maximizing prediction divergence between the target and tampered models, using meta-learning [52] to generalize across modification types. Experiments on BIVBench demonstrate MiSentry's superior detection performance, especially in detecting subtle tampering and benign modifications. Our contributions include:

- Exploring stricter black-box integrity verification by defining observable modifications and considering subtle tampering and benign modifications.
- Introducing *BIVBench*, the first benchmark for black-box integrity verification of DNN models, with 16 types of observable modifications, 10 more than previously used.
- Proposing *MiSentry*, a novel fingerprinting method leveraging subtle model modifications, meta-learning, and maximizing prediction divergence between the target model and subtly modified models in the model zoo to generate highly sensitive, generalizable, and effective fingerprint samples.
- Extensive experimental evaluation on BIVBench showing MiSentry's superior performance in detecting both seen and unseen observable modifications. MiSentry consistently outperforms all existing methods, particularly in detecting subtle modifications where existing state-of-the-art (SOTA) fingerprint methods are less effective or even ineffective.

2 Background: Black-box Integrity Verification

Both watermarking [54] and fingerprinting [17] methods are used for black-box integrity verification of DNN models through querying them with top-1 category labels.

Sensitive-Sample Fingerprinting (SSF) [17] is the first fingerprinting method to detect tampering with DNN models. It generates natural-looking fingerprint samples by approximating and maximizing their sensitivity to tampering and selects those with maximal neuron coverage. SSF's optimization problem is also solved with symbolic constraint solvers [10] and Bayesian optimization (BO) [28]. PublicCheck [48] uses generative models to create samples near the decision boundary, as this is sensitive to tampering.

Existing methods focus on detecting backdoor attacks and model compression but overlook subtler tampering. *Subtle model tampering* involves finer changes to the decision boundary than those addressed by current methods. BIVBench includes various types of subtle modifications, such as sample-level attacks (Targeted Attack, Degradation-S) and benign ones like online learning, unlearning, and fine-tuning (see Section 3.1 for details), for which current methods are less effective in detecting.

Model hashing [3, 22, 39] and watermarking [2] also verify integrity, but model hashing requires white-box access, contrasting with our black-box approach, while watermarking tampers with the model, potentially degrading performance, and cannot generate additional watermark samples post-training.

Black-box integrity verification allows live and stealthy checks during service, making it difficult for dishonest providers to evade detection. It is more practical and harder to evade, but also more challenging because only top-1 labels are available, complicating the detection of subtle changes.

Fingerprinting is also used for intellectual property (IP) protection of DNN models [4, 35, 49, 50]. IP protection verifies ownership of a stolen model and requires robustness against modifications, which involves different objectives and characteristics compared to integrity verification.

3 BIVBench

Traditional integrity verification detects all modifications, whether malicious or benign. Existing black-box integrity verification methods primarily focus on significant tampering, such as backdoor attacks, Trojan attacks, and model pruning, while overlooking subtle tampering and benign modifications. To address this gap, we introduce *BIVBench*, the first benchmark for black-box integrity verification of DNN models, designed to detect *any observable modifications* across a wide range of scenarios.

3.1 Practical Model Modifications to DNNs

BIVBench includes the following 16 types of benign and malicious observable model modifications for a comprehensive evaluation of black-box integrity verification methods.

3.1.1 *Benign Modifications.*

- **Unlearning** [12] makes a model forget specific training data, crucial for privacy compliance.
- **Online learning** [41] updates a model with new data.
- **Model compression** [7] reduces model size via:

Table 1: Tamper detection rate (%) with N_S fingerprint samples. "N/A" indicates data not available. For fine-tuning and retraining, "Last" or "All" specifies if only the last layer or all layers are fine-tuned, with numbers indicating the learning rate exponent (e.g., 10^{-3}). For degradation attacks, "S" denotes single-sample poisoning and "C" denotes poisoning all samples in a category, with "Random" or "Specific" indicating the type of label change.

Dataset	CIFAR-10						GTSRB						ImageNet					
Method	SSF		PubCheck		MiSentry		SSF		PubCheck		MiSentry		SSF		PubCheck		MiSentry	
Tampering Type \ N_S	1	5	1	5	1	5	1	5	1	5	1	5	1	5	1	5	1	5
Unlearning	3.2	20.9	28.3	86.6	**68.4**	**99.8**	1.9	16.4	19.9	73.5	**59.4**	**97.8**	39.6	94.1	56.3	97.8	**87.7**	**100.0**
Online Learning	6.7	36.8	25.4	82.3	**74.1**	**100.0**	4.2	35.7	21.8	69.7	**63.7**	**99.5**	46.1	96.3	48.2	96.9	**89.8**	**100.0**
Pruning	66.3	99.7	69.5	100.0	**89.5**	**100.0**	57.1	97.5	74.6	100.0	**84.5**	**100.0**	64.8	98.3	74.3	100.0	**92.4**	**100.0**
Quantization	66.7	99.8	60.8	99.4	**90.7**	**100.0**	78.5	100.0	71.5	100.0	**91.2**	**100.0**	84.5	100.0	81.9	100.0	**95.3**	**100.0**
Knowledge Distillation	78.4	100.0	69.2	100.0	**92.8**	**100.0**	76.2	99.9	68.3	100.0	**88.4**	**100.0**	82.7	100.0	88.7	100.0	**94.9**	**100.0**
Transfer Learning	85.9	100.0	74.3	100.0	**92.3**	**100.0**	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
Fine-tuning$_{\text{Last}}$-3	4.6	36.1	27.9	89.4	**73.7**	**100.0**	2.2	18.3	18.6	71.4	**61.1**	**99.2**	44.9	95.2	47.1	96.3	**88.1**	**100.0**
Fine-tuning$_{\text{Last}}$-4	1.8	15.3	24.8	83.2	**58.1**	**98.8**	0.5	9.1	16.8	66.9	**42.5**	**97.1**	35.5	92.7	41.3	91.8	**83.4**	**100.0**
Fine-tuning$_{\text{Last}}$-5	0.0	0.0	21.6	79.9	**45.9**	**96.2**	0.0	0.0	16.1	67.2	**36.8**	**90.3**	31.1	89.5	39.9	92.4	**82.3**	**99.8**
Fine-tuning$_{\text{All}}$-3	7.3	38.2	29.5	88.5	**76.5**	**100.0**	5.4	37.5	21.2	78.5	**64.9**	**99.5**	47.2	97.1	54.4	98.9	**90.7**	**100.0**
Fine-tuning$_{\text{All}}$-4	2.4	19.6	27.1	86.8	**64.1**	**99.4**	1.7	14.6	17.4	69.3	**58.2**	**96.4**	37.4	93.5	45.8	96.1	**87.5**	**100.0**
Fine-tuning$_{\text{All}}$-5	0.0	0.0	26.2	85.3	**59.2**	**97.6**	0.0	0.0	17.2	65.8	**44.5**	**94.8**	35.8	90.6	40.5	92.7	**83.2**	**100.0**
Retraining$_{\text{Last}}$	66.5	99.7	39.1	91.7	**84.5**	**100.0**	59.3	95.8	58.5	98.4	**78.1**	**99.9**	67.4	99.8	78.1	100.0	**93.6**	**100.0**
Retraining$_{\text{All}}$	76.1	100.0	65.4	99.3	**92.6**	**100.0**	64.5	97.3	64.8	99.1	**79.6**	**100.0**	68.0	100.0	82.0	100.0	**94.0**	**100.0**
Different Architectures	79.7	100.0	65.1	99.2	**93.5**	**100.0**	67.4	99.2	68.7	100.0	**81.9**	**100.0**	78.3	100.0	84.5	100.0	**95.1**	**100.0**
Degradation$_{\text{Random}}$-C	43.9	92.5	48.4	94.7	**91.3**	**100.0**	49.6	96.5	53.5	95.8	**85.3**	**100.0**	61.3	97.4	73.4	99.9	**91.5**	**100.0**
Degradation$_{\text{Specific}}$-C	46.2	95.2	47.9	94.5	**93.1**	**100.0**	55.3	97.4	59.2	97.9	**92.7**	**100.0**	64.5	98.9	72.9	100.0	**91.6**	**100.0**
Degradation-S	19.8	72.4	38.7	91.1	**88.2**	**100.0**	26.1	85.7	38.9	92.2	**83.4**	**100.0**	56.9	96.8	67.3	99.8	**92.3**	**100.0**
Targeted Attack	11.4	63.5	36.9	90.2	**83.8**	**100.0**	12.4	67.1	42.1	94.4	**79.4**	**100.0**	48.1	94.9	63.8	99.8	**90.9**	**100.0**
Bit-flipping	84.0	100.0	91.0	100.0	**91.0**	**100.0**	89.0	100.0	88.0	100.0	**92.0**	**100.0**	96.0	100.0	96.0	100.0	**98.0**	**100.0**
Backdoor	64.9	98.8	73.6	99.6	**86.4**	**100.0**	63.2	98.6	69.3	99.7	**85.8**	**100.0**	65.6	99.8	81.4	100.0	**92.1**	**100.0**
Trojan	74.3	100.0	78.8	100.0	**91.9**	**100.0**	71.6	99.8	71.1	99.5	**87.1**	**100.0**	77.5	100.0	79.6	100.0	**93.4**	**100.0**
Clean label	72.4	100.0	79.8	100.0	**92.1**	**100.0**	69.8	100.0	81.5	100.0	**89.4**	**100.0**	78.4	100.0	84.2	100.0	**92.9**	**100.0**

(i) **Pruning** [33] removes less critical components.
(ii) **Quantization** reduces precision to save resources.

- **Knowledge distillation** [18] transfers knowledge from a larger model to a smaller one.
- **Transfer learning** [34] adapts a pre-trained model for new tasks.
- **Fine-tuning** adjusts weights in specific layers.
- **Retraining** re-initializes and re-trains tunable layers' weights.

3.1.2 Malicious Modifications.

- **Poisoning degradation attack** [21] degrades performance using tampered training data.
- **Bit-flipping attack** [37] degrades performance by flipping vulnerable bits.
- **Targeted attack** [24, 42, 45] causes mispredictions for specific inputs but normal predictions otherwise.
- **Backdoor attack** [14, 23] causes mispredictions on triggered inputs while performing normally on others.
- **Trojan attack** [30] injects a backdoor with optimized triggers.
- **Clean-label attack** [53] inserts a backdoor using visually clean poisoned data.

3.2 Fingerprinting Evaluation Using BIVBench

We evaluate the performance of existing SOTA fingerprinting methods using BIVBench. Performance is measured by the tampering detection rate with N_S fingerprint samples per tampering type. We use CIFAR-10 [25], GTSRB [44], and ImageNet [40] datasets with ResNet20 [15], a 6-layer CNN [29], and DenseNet121 [20], respectively. Test instances are constructed with *subtlest modification*: stopping modification once the objective is met. We assess two SOTA methods, SSF [17] and PublicCheck [48], along with our method, MiSentry (described in Section 4). Detection rates with $N_S = 1$ and $N_S = 5$ samples are shown in Table 1. Detection rates are notably lower for subtler modifications, such as unlearning, online learning, fine-tuning, targeted attacks, and single-sample poisoning, compared to more obvious tampering like model compression and backdoor attacks. For instance, SSF fails to detect fine-tuning with a learning rate of 10^{-5}. This underscores the inadequacy of current methods in detecting subtle tampering and highlights the need for more sensitive fingerprinting approaches.

4 MiSentry: A Sensitive Fingerprinting Method

We introduce MiSentry, a fingerprinting method designed for enhanced detection sensitivity. We outline the threat model and desirable properties of fingerprint samples. To achieve these properties, we frame fingerprint generation as an optimization problem, select representative models for a model zoo, and use meta-learning to ensure effectiveness across various model modifications. Figure 1 illustrates MiSentry's fingerprint generation process.

4.1 Threat Model

MiSentry assumes white-box access for fingerprint generation and black-box access for integrity verification, similar to SSF [17]. Fingerprints are generated with full model access, but verification is limited to querying for the top-1 label. MiSentry supports both private and public verification. In public settings [48], generated fingerprint samples are managed by a trusted third party and are accessible to any interested party. In private settings, only authorized users can access and verify with the samples. While public

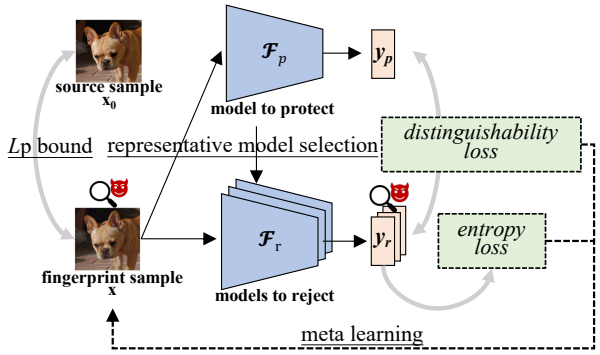


Figure 1: Overview of MiSentry's fingerprint generation

verification is more practical, it increases the risk of sample leakage, allowing adversaries to adapt their models to evade detection. Consequently, we assume adversaries may access some fingerprint samples and exploit them to bypass integrity checks. Additionally, we consider the DNN service provider to be untrustworthy, with the potential to detect and evade fingerprint queries.

4.2 Desirable Properties

Fingerprint samples should exhibit the following properties:

- **Sensitivity.** Detect any observable modifications to the model.
- **Uniqueness.** Distinguish the target model from others, even if they share the same architecture, dataset, and training process.
- **Covertness.** Ensure both input and output covertness, so fingerprint queries resemble normal ones and avoid detection. Both SSF [17] and PublicCheck [48] require input covertness but not output covertness.
- **Robustness to partial leakage.** Remain effective even if some fingerprint samples are leaked.
- **Generalizability.** Detect various practical modifications, both seen and unseen.

4.3 Formalizing as an Optimization Problem

- **Natural appearance.** Input covertness requires that a fingerprint sample x looks natural. This is achieved by constraining its distortion to a normal sample x_0 within ϵ: $||x - x_0||_p \leq \epsilon$ for some norm p.
- **Distinguishability loss.** To distinguish the target model $\mathcal{F}_p$ from another model $\mathcal{F}_r$, we maximize the divergence between their output predictions over a fingerprint sample x, measured by Kullback-Leibler (KL) divergence. The corresponding loss is:

$$\begin{aligned}\ell_D(x, \mathcal{F}_p, \mathcal{F}_r) &= -D_{KL}(\mathcal{F}_p(x)||\mathcal{F}_r(x)) \\ &= \sum_j \mathcal{F}_p(x)_j \ln \frac{\mathcal{F}_r(x)_j}{\mathcal{F}_p(x)_j}\end{aligned} \tag{1}$$

where $\mathcal{F}_p(x)_j$ is the j-th element in the prediction vector of $\mathcal{F}_p$ on input x. The negative sign indicates that we maximize this loss.

- **Entropy loss.** A normal sample typically has a high probability for its true label and low probabilities for others, resulting in a low entropy prediction vector. A fingerprint sample x should mimic this behavior. To ensure output covertness on both $\mathcal{F}p$ and $\mathcal{F}r$, we minimize the information entropy H of the prediction vector on

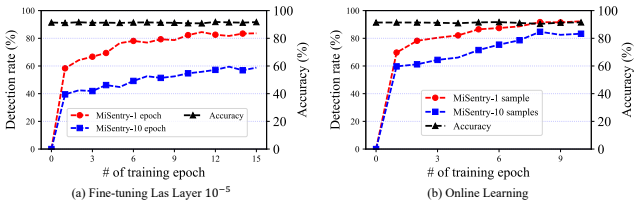


Figure 2: Tamper detection rate w.r.t. the number of training epoch/samples on CIFAR10

both models:

$$\ell_{E_\gamma}(x, \mathcal{F}\gamma) = \mathrm{H}(\mathcal{F}\gamma(x)) = -\sum_j \mathcal{F}\gamma(x)j \log \mathcal{F}\gamma(x)j \tag{2}$$

where $\gamma \in \{p, r\}$ for models $\mathcal{F}_p$ and $\mathcal{F}_r$, respectively. The entropy loss of x should lie within a specified range $[\alpha_\gamma, \beta_\gamma]$ on both models, e.g., 10th percentile to 90th percentile of normal samples.

- **Optimization problem.** A fingerprint sample x is generated by solving the following optimization problem:

$$\begin{aligned}&\min_x \ell_{total}(x, \mathcal{F}_p, \mathcal{F}_r) \\ &\text{s.t. } ||x - x_0||_p \leq \epsilon, \alpha_p \leq \ell_{E_p} \leq \beta_p, \alpha_r \leq \ell_{E_r} \leq \beta_r\end{aligned} \tag{3}$$

where

$$\begin{aligned}\ell_{total}(x, \mathcal{F}_p, \mathcal{F}_r) =& \ell_D(x, \mathcal{F}_p, \mathcal{F}_r) + \lambda_1 \cdot \ell_{E_p}(x, \mathcal{F}_p) \\ &+ \lambda_2 \cdot \ell_{E_r}(x, \mathcal{F}_r),\end{aligned} \tag{4}$$

λ_1 and λ_2 are weighting hyperparameters. This optimization problem can be solved iteratively using projected gradient descent (PGD) [26, 27]:

$$x_{k+1} = Clip_\epsilon(x_k - \eta \cdot \nabla_{x_k} \ell_{total}(x_k, \mathcal{F}_p, \mathcal{F}_r)) \tag{5}$$

- **Generation diversity.** Fingerprint samples are generated from various source samples to ensure diversity and broad distribution across the model's decision boundary. Each sample occupies a unique local region, which minimizes the impact of leaked samples on the effectiveness of the remaining ones.

4.4 Representative Model Selection

This section describes how MiSentry selects representative models to maximize detection sensitivity while minimizing the number of models used for fingerprint generation. To ensure practicality, models in the meta-learning model zoo should have the same architecture and dataset as the target model, facilitating their generation during the target model's training.

In *support vector machines* (SVMs) [47], support vectors are critical instances that define the decision boundary. Similarly, we identify key modified model instances to generate effective fingerprint samples.

Our approach is based on the intuition that subtler modifications are more challenging to distinguish from the target model due to their closer decision boundaries. Therefore, fingerprint samples from subtler modifications are expected to be more sensitive than those from more significant modifications. We validate this by generating fingerprint samples from models fine-tuned for 1 and 10 epochs and testing them against models modified by 1 to 15 epochs. Similarly, fingerprint samples from models updated with 1 and 10 samples were tested against models updated with up to 10

samples. Results on CIFAR-10, shown in Fig. 2, confirm that subtler modifications yield more sensitive fingerprint samples.

Consequently, we use models with subtler modifications as "support vectors" in our model zoo. For different modification types, we empirically select models that cause fewer perturbations to the decision boundary, focusing on those with the same architecture and dataset as the target model. This ensures practical fingerprint sample generation without needing additional data. Details of the selected models are described in Section 5.1.2. Additionally, leveraging meta-learning [52] for generalizability ensures that MiSentry's limited model selection does not compromise the effectiveness of fingerprint samples on unseen modifications. As shown in Table 1, MiSentry-generated fingerprints effectively distinguish the target model from both models in the model zoo and unseen models.

5 Experimental Evaluation

We conduct an extensive evaluation of MiSentry using BIVBench and compare it against two SOTA baselines: SSF [17] and PublicCheck [48].

5.1 Experimental Settings

5.1.1 Fingerprint Generation. For SSF, we use the official open-source code [16] with default settings. SSF employs the L_2-norm to bind a fingerprint sample to a normal sample, and we apply the same L_2-norm in Eq. 3 for MiSentry to ensure a fair comparison. For PublicCheck, we generate fingerprint samples near the decision boundary using VQVAE[46]. We train VQVAEs on the CIFAR-10 and GTSRB training sets using Pythea [5] and use a pre-trained VQVAE from [11] for ImageNet.

5.1.2 Modified Models.

• **Modification settings**. We construct BIVBench by setting each modification type to be as difficult as possible to distinguish from the target model. Modifications are made as subtle as possible by stopping once the modification goal is achieved. The learning rate is set to 1.0×10^{-4}, with a batch size of 64 for all datasets. Detailed settings for each modification type are provided in the supplementary materials.

• **Model zoo**. The model zoo in our meta-learning process includes three benign modifications as rejected models: fine-tuning the last layer and all layers at a learning rate of 10^{-5}, and random start (retraining all layers). Five models are generated for each modification, except for ImageNet, where we use a DenseNet121 model from an independent public zoo [9] due to the time required to train from scratch.

• **Test models**. Test models are independent of those in the model zoo. We generate 10 models for each modification type, except for the deterministic bit-flipping attack, where only one model is generated. For different architectures, we use 10 architectures [8, 15, 19, 20, 43], as detailed in the supplementary materials, and source pre-trained models from [36] for ImageNet and from [6] for CIFAR-10. GTSRB models are trained from scratch due to the lack of publicly available pre-trained models.

5.1.3 Tamper Detection Setting.

• **Tamper detection**. Model integrity is verified by querying with a set of fingerprint samples, $\mathcal{S}_F$, and comparing the returned top-1 labels with the target model's labels, which serve as the ground truth. A model is flagged as modified if any fingerprint sample produces a mismatched label.

• **Fingerprint selection**. For MiSentry and PublicCheck, we generate 100 fingerprint samples. For SSF, we generate 2000 sensitive samples and select 100 with the highest active neuron coverage as fingerprint samples. For each tamper detection, we randomly select N_S samples from these 100.

• **Tamper detection rate.** The detection process is repeated 100 times for each model, using independent sets of N_S fingerprint samples each time. The detection rate is the ratio of detected models to the total number of tests. For instance, the rate for unlearning is calculated as detected models out of 1000 (= 10 test models × 100 detection runs).

5.2 Tampering Detection Performance

Table 1 in Section 3.2 compares the tampering detection performance of MiSentry, SSF, and PublicCheck. Key observations include:

• **Generalizable fingerprint samples by MiSentry**. Despite using a limited set of model modifications in its meta-learning model zoo, MiSentry effectively detects unseen tampering types and independently trained models. Detection rates exceed 36% with a single fingerprint and 90% with five fingerprints across all three datasets.

• **MiSentry outperforms existing methods**. MiSentry achieves robust detection rates, even for subtle tampering types. It not only outperforms both SSF and PublicCheck overall but also demonstrates a significant advantage in detecting more challenging modifications.

• **Higher detection rates on higher-resolution images**. Detection rates for subtler tampering types, such as unlearning, online learning, fine-tuning, poisoning degradation, and targeted attacks, are generally higher on ImageNet than on CIFAR-10 and GTSRB for all methods. This is likely because one-epoch fine-tuning of an ImageNet model (with its larger dataset) introduces more noticeable modifications than that on CIFAR-10 or GTSRB. Additionally, modifications on the larger ImageNet models involve more parameters, making them easier to detect.

5.3 Detection Sensitivity

To assess how quickly a fingerprinting method can detect manipulations of the target model, we examine detection sensitivity—specifically, the relationship between tampering detection rate and the magnitude of modification from the initial unmodified model. Fig. 3 presents the tampering detection rates of different fingerprinting methods at various training steps on CIFAR-10, focusing on fine-tuning the last layer with a learning rate of 10^{-5}, targeted attacks, and backdoor attacks. Key observations include:

• **Rapid detection with MiSentry**. MiSentry's detection rate increases more rapidly compared to SSF and PublicCheck, indicating higher detection sensitivity.

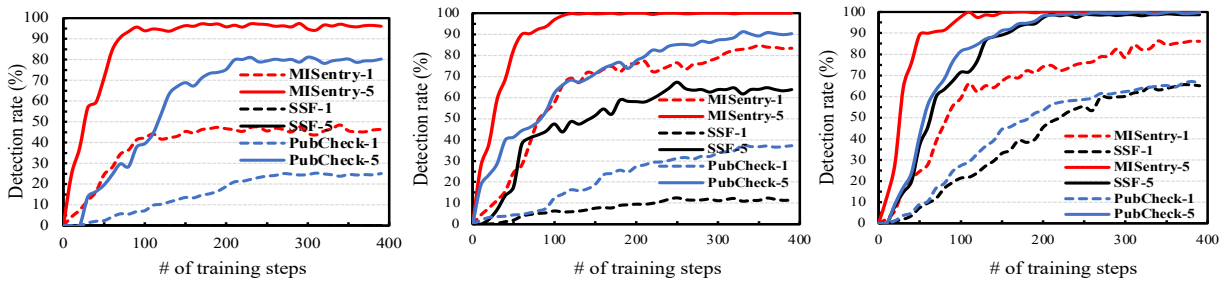
• **Early saturation of MiSentry's detection performance**. When using 5 fingerprint samples, MiSentry's detection performance saturates around 100 training steps, much earlier than SSF and PublicCheck.

• **SSF's limitation**. SSF fails to detect fine-tuning, consistent with the results reported in Table 1 in Section 3.2.

5.4 Perceptual Quality and Output Covertness

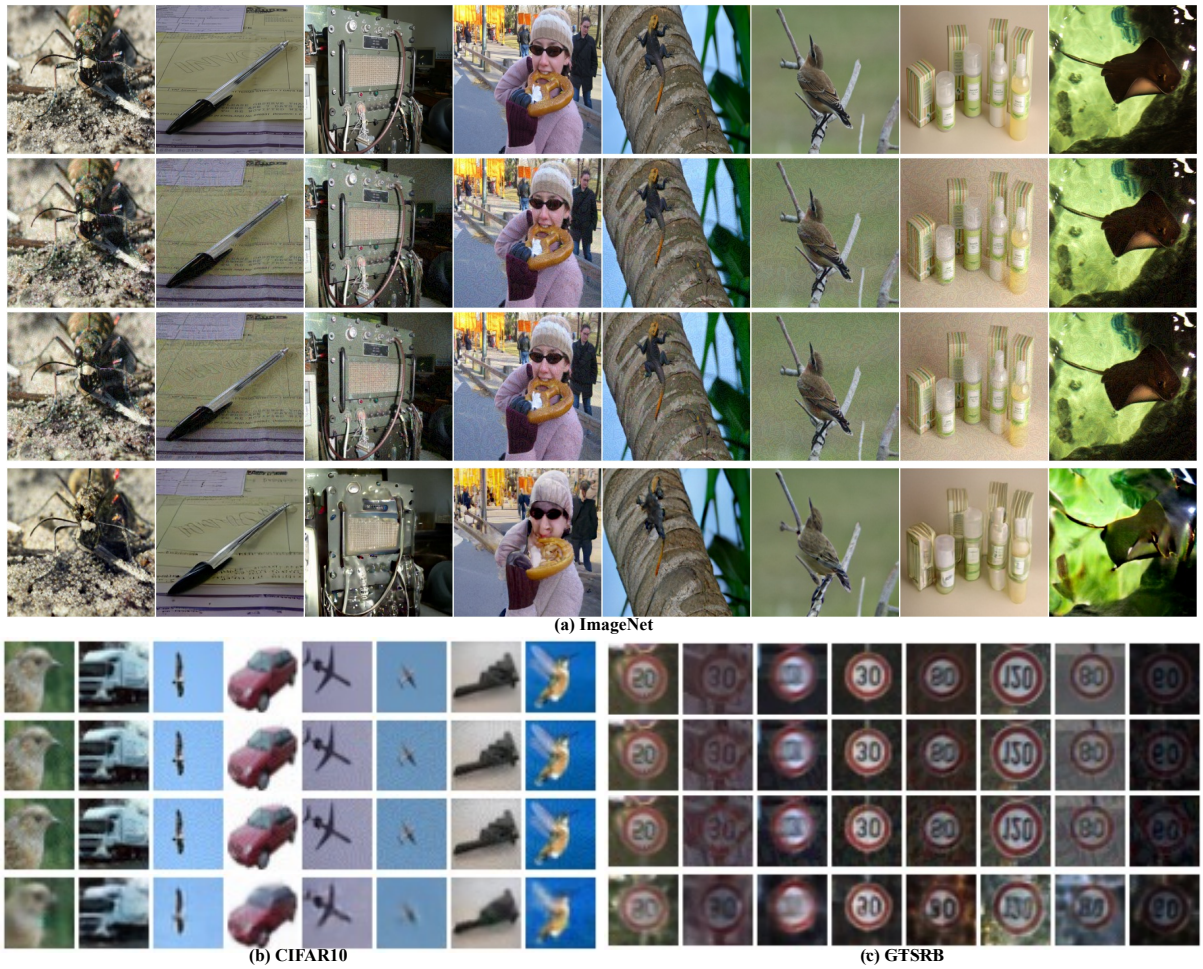
5.4.1 Perceptual Quality. Figs. 4 shows fingerprint samples generated by MiSentry, SSF, and PublicCheck on ImageNet, CIFAR-10, and GTSRB. MiSentry and SSF produce nearly imperceptible, noise-like artifacts that preserve the natural appearance of the images, while PublicCheck's artifacts noticeably alter texture and color temperature.

5.4.2 Output Covertness. To avoid detection, the output of a fingerprint sample should be indistinguishable from that of a normal



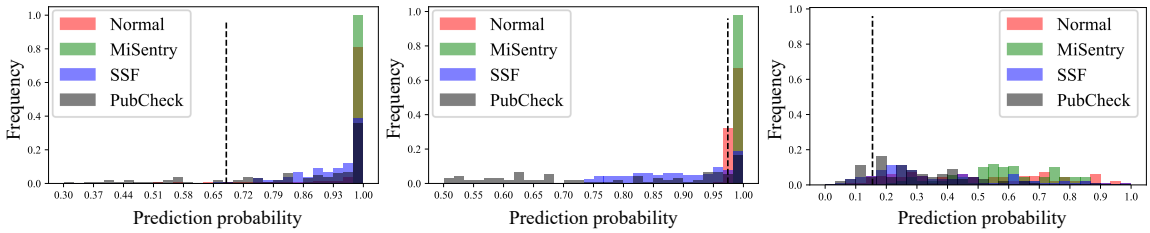
(a) Fine-tune last layer 10^{-5} (b) Targeted attack (c) Backdoor attack

Figure 3: Tamper detection rate vs the number of training steps



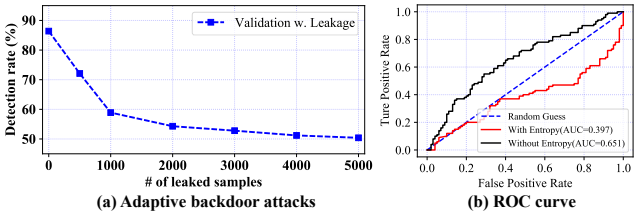
(a) ImageNet (b) CIFAR10 (c) GTSRB

Figure 4: Perceptual quality of fingerprint samples generated on ImageNet, CIFAR-10, and GTSRB. From top to bottom, each row represents source images, and fingerprint images generated by MiSentry, SSF, and PublicCheck, respectively.



(a) CIFAR10 (b) GTSRB (c) ImageNet

Figure 5: Distributions of prediction probabilities of top-1 labels for normal samples and fingerprint samples



(a) Adaptive backdoor attacks (b) ROC curve

Figure 6: (a) Detection rates using varying numbers of fingerprint samples in adaptive backdoor attacks against MiSentry on CIFAR-10. (b) ROC curves of fingerprint detection based on low-confidence predictions.

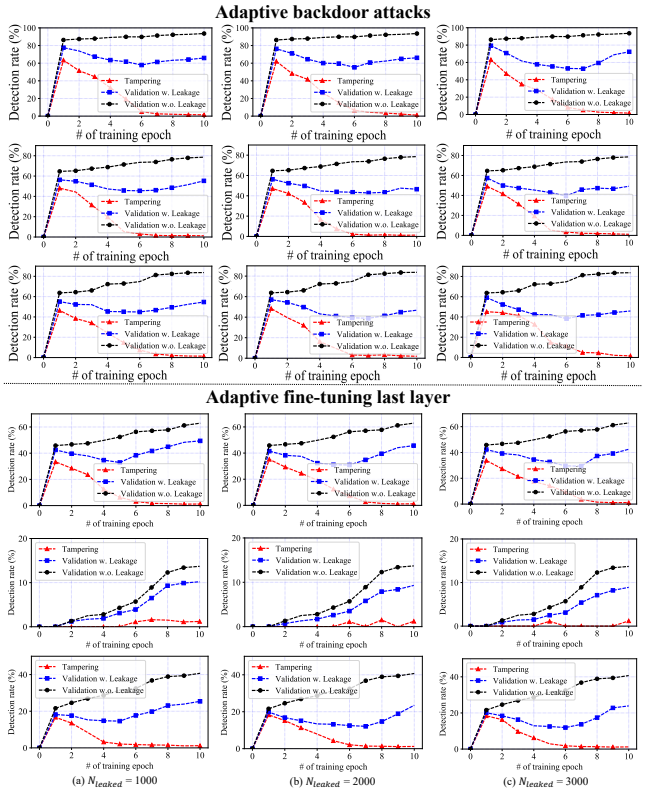
sample. Fig. 5 shows the distribution of top-1 prediction probabilities for fingerprint samples generated by MiSentry, SSF, and PublicCheck, compared to normal samples in the test set. MiSentry's prediction probabilities align closely with those of normal samples, while some of SSF's and PublicCheck's predictions fall below the bottom 5% confidence level of normal samples (indicated by dashed lines), particularly for GTSRB. This makes fingerprint samples from SSF and PublicCheck more likely to be singled out by the model service provider. Among the three methods, only MiSentry meets the output covertness requirement.

5.5 Adaptive Attacks with Leaked Fingerprints

When MiSentry is deployed, an adversary might collect fingerprint samples—either from previously used ones, by requesting them from the fingerprint sample distributor for public verification, or by generating new fingerprints with MiSentry if the target model is accessible. The adversary could then attempt an adversarial tampering attack by manipulating the model while ensuring that the top-1 labels for the collected fingerprint samples remain unchanged.

To evaluate MiSentry's resilience to such adaptive attacks, we increase the number of fingerprint samples used in these attacks. Fig. 6(a) shows the detection rate for adaptive backdoor attacks on CIFAR-10 using a single fingerprint sample. The detection rate decreases from 86.4% to 50.4% as the number of exploited fingerprint samples increases from 0 to 5000. However, the detection rate appears to stabilize around 50% with further exploitation, demonstrating MiSentry's robustness against adaptive tampering attacks.

Figure 7 compares MiSentry, SSF, and PublicCheck under adaptive tampering attacks on CIFAR-10. The black dotted line represents the detection rate when no leaked fingerprint samples are exploited, while the blue square line indicates the detection rate when N_{leaked} fingerprint samples are used in the attack. The red triangle line shows the detection rate when leaked fingerprint samples are employed for tampering detection. With increasing training



(a) N_{leaked} = 1000 (b) N_{leaked} = 2000 (c) N_{leaked} = 3000

Figure 7: Detection rates when different numbers of fingerprint samples are leaked and exploited for adaptive tampering attacks on CIFAR-10. From top to bottom, each row shows the results of MiSentry, SSF, and PublicCheck, respectively.

epochs, the red triangle line gradually decays to a low detection rate near 0, indicating leaked fingerprints are effectively evaded, achieving the goal of the adaptive tampering attacks. Yet the blue square line of MiSentry remains high, indicating its robustness to adaptive tampering attacks. With N_{leaked} = 1000, MiSentry's lowest detection rate is 1.28× that of SSF and 1.31× that of PublicCheck under adaptive backdoor attacks. This advantage persists even when N_{leaked} = 3000. In the case of adaptive fine-tuning, MiSentry's detection rate advantage over PublicCheck increases to 2.25×. MiSentry consistently maintains a higher detection rate than both SSF and PublicCheck across all scenarios.

MiSentry's robust detection performance against adaptive tampering attacks is due to the diversity of its generated fingerprint samples. By starting from various source samples, MiSentry produces a broad range of fingerprint samples widely distributed across

Table 2: Average time cost per model construction or fingerprint sample generation (in seconds)

		CIFAR-10	GTSRB	ImageNet
Modified model	Fine-tuning$_{\text{Last}}$	9.5	10.4	4980.3
	Fine-tuning$_{\text{All}}$	11.2	12.8	6361.5
	Retraining$_{\text{All}}$	1473.6	1519.8	N/A
Fingerprint sample generation		95.7	86.1	925.2

the decision boundary of the target model. Each fingerprint sample is independent and unpredictable, occupying a unique local region, which makes it difficult for leaked fingerprint samples to compromise the effectiveness of the remaining ones.

5.6 Time Cost

Our experiment was conducted using a single Nvidia RTX 3090 GPU. Table 2 presents the average time required to construct a model for each tampering type and to generate fingerprint samples using MiSentry. As expected, the time cost increases with the model's parameter size. However, these time costs are justified by the substantial benefits provided by its integrity verification.

5.7 Ablation Study

We conducted an ablation study on CIFAR-10 to evaluate the impact of each component of MiSentry. The tamper detection rates using a single fingerprint sample ($N_S = 1$) generated by MiSentry are reported below.

5.7.1 Modification Types in Model Zoo. We first examine the effect of different modification types in MiSentry's model zoo. Table 3 shows the tampering detection rates for various model zoo configurations. The results indicate that subtly modified models, such as last-layer fine-tuning, are crucial for detecting all modification types, while other modification types primarily offer incremental detection improvements.

5.7.2 Number of Models per Type in the Model Zoo. To explore the relationship between the generalization ability of fingerprint samples and the number of minimally modified models per type in the model zoo, we conducted an ablation study by incrementally increasing the number of fine-tuned last-layer models. As shown in Table 3, the generalization ability of fingerprint samples improves with the number of representative models per modification type but saturates when this number exceeds 6.

Table 3: Ablation study on modification types and model counts per type in the model zoo. RT: Retrain, FT: fine-tuning (FT includes FT_{Last} + FT_{All} unless specified).

Modification	Model Zoo Combination				Number of Models/Type			
	$FT + RT$	FT	FT_{Last}	FT_{All}	2	4	6	8
Degradation-C	91.3	91.1	88.9	79.4	60.1	90.1	91.5	90.4
Targeted Attack	83.8	84.2	79.3	77.2	59.6	82.2	85.3	84.6
Backdoor	86.4	85.9	81.5	78.1	61.3	83.9	88.5	87.8
Quantization	90.7	91.2	89.4	80.1	68.3	91.4	91.8	92.2
Pruning	89.5	87.4	78.3	72.4	60.2	88.7	88.9	89.2
Fine-tuning$_{\text{Last}}$	73.7	73.8	75.8	48.9	24.3	39.8	47.5	49.1
Fine-tuning$_{\text{All}}$	59.2	59.4	49.2	48.1	24.2	48.5	58.7	60.2

Table 4: Ablation study on target model architecture

Tampering Type	Target Model's Architecture		
	Resnet-20	VGG11	ViT
Targeted Attack	83.8	79.4	85.7
Backdoor	86.4	88.5	88.3
Fine-tuning$_{\text{Last}}$	45.9	51.8	51.3
Fine-tuning$_{\text{All}}$	59.2	57.3	64.7

5.7.3 Effect of L_p Bound on Input Covertness and Detection Performance. We then evaluate the effect of the L_p norm bound between a fingerprint sample and its source sample. Without the L_p norm bound, the generated fingerprints become noise-like images, easily distinguishable from normal samples but with higher detection sensitivity. For instance, in the most challenging case—fine-tuning the last layer with a learning rate of 10^{-5}—the tampering detection rate improves by 0.359× without the L_p norm bound.

5.7.4 Effect of Entropy Loss on Output Covertness. Entropy loss ensures that the generated fingerprint samples produce prediction vectors similar to those of normal samples. Without entropy loss, fingerprint samples may exhibit abnormal prediction vectors and be identified by their confidence scores, which facilitates evading MiSentry. We conducted an ablation study on entropy loss, detecting low-confidence samples as potential fingerprint samples. The results on CIFAR-10 are shown in Fig. 6(b). The AUC scores were 0.651 without entropy loss and 0.397 with it, confirming that entropy loss enhances the covertness of fingerprint prediction vectors. This makes the entropy constraint a crucial component of MiSentry, allowing it to evade fingerprint detection based on output probability distributions.

5.7.5 Effect on Target Model Architectures. To assess MiSentry's generalizability across different model architectures, we conducted experiments using VGG-11 and ViT as the target models. As shown in Table 4, MiSentry demonstrates consistent detection performance across these architectures, confirming its generalizability.

6 Conclusion

This paper presents significant advancements in *black-box integrity verification* (BIV) for DNN models. We introduce a more stringent form of BIV, focusing on detecting subtle modifications and establishing observable changes, thereby setting a new standard in the field. To support this, we develop BIVBench, the first benchmark specifically designed for stricter BIV. BIVBench includes 16 modification types—10 more than previous studies—covering both subtle and significant tampering, as well as benign changes, providing comprehensive coverage of model modification scenarios. We use BIVBench to highlight the limitations of existing fingerprinting methods. Additionally, we introduce MiSentry, a novel fingerprinting approach that leverages subtly tampered models to generate highly sensitive and generalizable fingerprint samples, effectively detecting both seen and unseen modifications. MiSentry outperforms existing *state-of-the-art* methods overall and significantly surpasses them in detecting subtle modifications. In summary, we establish new benchmarks and methodologies for BIV of DNN models, paving the way for future research in this critical area.

Acknowledgements

This work was partially supported by National Natural Science Foundation of China under Grant No.62272175.

References

[1] Amazon Web Services. 2023. Amazon SageMaker. https://aws.amazon.com/sagemaker.
[2] Marco Botta, Davide Cavagnino, and Roberto Esposito. 2021. NeuNAC: A novel fragile watermarking algorithm for integrity protection of neural networks. *Information Sciences* 576 (2021), 228–241.
[3] James H. Burrows. 1995. Secure hash standard. *Federal information processing standards publication* (1995), 180–1.
[4] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2021. IPGuard: Protecting Intellectual Property of Deep Neural Networks via Fingerprinting the Classification Boundary. In *Proceedings of the ACM Asia Conference on Computer and Communications Security, ASIA CCS '21*. ACM, 14–25.
[5] Clément Chadebec, Louis Vincent, and Stephanie Allassonniere. 2022. Pythae: Unifying Generative Autoencoders in Python - A Benchmarking Use Case. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Curran Associates, Inc., 21575–21589.
[6] Yaofo Chen. 2022. Pytorch CIFAR Models. https://github.com/chenyaofo/pytorch-cifar-models.
[7] Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. 2017. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282* (2017).
[8] François Chollet. 2017. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 1251–1258.
[9] Distributed (Deep) Machine Learning Community. 2021. Gluon CV Toolkit. https://github.com/dmlc/gluon-cv.
[10] Amel Nestor Docena, Thomas Wahl, Trevor Pearce, and Yunsi Fei. 2021. Sensitive Samples Revisited: Detecting Neural Network Attacks Using Constraint Solvers. In *Proceedings of the 9th International Symposium on Symbolic Computation in Software Science, SCSS 2021 (EPTCS, Vol. 342)*. Research Institute for Symbolic Computation, 35–48.
[11] Patrick Esser, Robin Rombach, and Bjorn Ommer. 2021. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 12873–12883.
[12] Aditya Golatkar, Alessandro Achille, and Stefano Soatto. 2020. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 9304–9312.
[13] Google Cloud. 2023. AutoML. https://cloud.google.com/automl.
[14] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. Badnets: Evaluating backdooring attacks on deep neural networks. *IEEE Access* 7 (2019), 47230–47244.
[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 770–778.
[16] Zecheng He. 2021. The Implementation of Sensitive Sample Fingerprinting. https://github.com/zechenghe/Sensitive_Sample_Fingerprinting.
[17] Zecheng He, Tianwei Zhang, and Ruby Lee. 2019. Sensitive-sample fingerprinting of deep neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 4729–4737.
[18] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2014. Distilling the knowledge in a neural network. In *Proceedings of the Twenty-eighth Annual Conference on Neural Information Processing Systems (NIPS 2014) Deep Learning Workshop*. Curran Associates, Inc.
[19] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).
[20] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q. Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 4700–4708.
[21] Matthew Jagielski, Giorgio Severi, Niklas Pousette Harger, and Alina Oprea. 2021. Subpopulation data poisoning attacks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 3104–3122.
[22] Mojan Javaheripi and Farinaz Koushanfar. 2021. HASHTAG: Hash Signatures for Online Detection of Fault-Injection Attacks on Deep Neural Networks. In *Proceedings of the 2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
[23] Xue Jiang, Lubin Meng, Siyang Li, and Dongrui Wu. 2023. Active poisoning: efficient backdoor attacks on transfer learning-based brain-computer interfaces. *Science China Information Sciences* 66, 8 (2023), 1–22.
[24] Pang Wei Koh and Percy Liang. 2017. Understanding black-box predictions via influence functions. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1885–1894.
[25] Alex Krizhevsky. 2009. *Learning multiple layers of features from tiny images*. Citeseer.
[26] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. 2017. Adversarial examples in the physical world. In *Proceedings of the 5th International Conference on Learning Representations ICLR 2017, Workshop track*. OpenReview.net.
[27] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. 2017. Adversarial machine learning at scale. In *Proceedings of the 5th International Conference on Learning Representations ICLR 2017*. OpenReview.net.
[28] Deepthi Praveenlal Kuttichira, Sunil Gupta, Dang Nguyen, Santu Rana, and Svetha Venkatesh. 2022. Verification of integrity of deployed deep learning models using Bayesian Optimization. *Knowledge-Based Systems* 241, C (2022), 108238.
[29] Yann LeCun, Lawrence D. Jackel, Léon Bottou, Corinna Cortes, John S Denker, Harris Drucker, Isabelle Guyon, Urs A Muller, Eduard Sackinger, Patrice Simard, and Vladimir Vapnik. 1995. Learning algorithms for classification: A comparison on handwritten digit recognition. *Neural Networks* (1995), 261–276.
[30] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. 2018. Trojaning Attack on Neural Networks. In *Proceedings of the 25th Annual Network and Distributed System Security Symposium, NDSS 2018*. The Internet Society.
[31] Jonathan Long, Evan Shelhamer, and Trevor Darrell. 2015. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 3431–3440.
[32] Microsoft Azure. 2023. Microsoft Machine Learning. https://azure.microsoft.com/en-us/products/machine-learning.
[33] Pavlo Molchanov, Arun Mallya, Stephen Tyree, Iuri Frosio, and Jan Kautz. 2019. Importance estimation for neural network pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 11264–11272.
[34] Sinno Jialin Pan and Qiang Yang. 2010. A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering* 22, 10 (2010), 1345–1359.
[35] Xudong Pan, Yifan Yan, Mi Zhang, and Min Yang. 2022. MetaV: A Meta-Verifier Approach to Task-Agnostic Model Fingerprinting. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 1327–1336.
[36] Pytorch. 2022. Pytorch Hub. https://pytorch.org/hub/.
[37] Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. 2019. Bit-flip attack: Crushing neural network with progressive bit search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. IEEE, 1211–1220.
[38] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 779–788.
[39] Ronald Rivest. 1992. RFC1321: The MD5 message-digest algorithm.
[40] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision* 115, 3 (2015), 211–252.
[41] Ahmed Salem, Apratim Bhattacharya, Michael Backes, Mario Fritz, and Yang Zhang. 2020. Updates-Leak: Data Set Inference and Reconstruction Attacks in Online Learning. In *Proceedings of the 29th USENIX Security Symposium, USENIX Security 2020*. USENIX Association, 1291–1308.
[42] Ali Shafahi, W. Ronny Huang, Mahyar Najibi, Octavian Suciu, Christoph Studer, Tudor Dumitras, and Tom Goldstein. 2018. Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*. Curran Associates, Inc., 6106–6116.
[43] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *Proceedings of the 3th International Conference on Learning Representations, ICLR 2015*. OpenReview.net.
[44] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. 2012. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks* 32 (2012), 323–332.
[45] Octavian Suciu, Radu Marginean, Yigitcan Kaya, Hal Daume III, and Tudor Dumitras. 2018. When does machine learning FAIL? generalized transferability for evasion and poisoning attacks. In *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, 1299–1316.
[46] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. 2017. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. Curran Associates, Inc., 6309–6318.
[47] Vladimir Vapnik. 1999. *The nature of statistical learning theory*. Springer science & business media.

[48] Shuo Wang, Sharif Abuadbba, Sidharth Agarwal, Kristen Moore, Ruoxi Sun, Minhui Xue, Surya Nepal, Seyit Camtepe, and Salil Kanhere. 2022. PublicCheck: Public Integrity Verification for Services of Run-time Deep Models. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 1239–1256.

[49] Siyue Wang, Xiao Wang, Pin-Yu Chen, Pu Zhao, and Xue Lin. 2021. Characteristic Examples: High-Robustness, Low-Transferability Fingerprinting of Neural Networks. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI*. 575–582.

[50] Kang Yang, Run Wang, and Lina Wang. 2022. MetaFinger: Fingerprinting the Deep Neural Networks with Meta-training. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022*. ijcai.org, 776–782.

[51] Fan Yao, Adnan Siraj Rakin, and Deliang Fan. 2020. DeepHammer: Depleting the Intelligence of Deep Neural Networks through Targeted Chain of Bit Flips. In *Proceedings of the 29th USENIX Security Symposium, USENIX Security 2020*. USENIX Association, 1463–1480.

[52] Zheng Yuan, Jie Zhang, Yunpei Jia, Chuanqi Tan, Tao Xue, and Shiguang Shan. 2021. Meta gradient adversarial attack. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 7748–7757.

[53] Shihao Zhao, Xingjun Ma, Xiang Zheng, James Bailey, Jingjing Chen, and Yu-Gang Jiang. 2020. Clean-Label Backdoor Attacks on Video Recognition Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 14431–14440.

[54] Renjie Zhu, Ping Wei, Sheng Li, Zhaoxia Yin, Xinpeng Zhang, and Zhenxing Qian. 2021. Fragile Neural Network Watermarking with Trigger Image Set. In *International Conference on Knowledge Science, Engineering and Management*. Springer, 280–293.